Homework 2

Atharva Chalke

April 21, 2018

Problem 1. Figure DFA (in Files) shows a seven state DFA with the binary alphabet. Show each step of successive refinement algorithm that determines which pairs of states are distinguishable and which are indistinguishable. You have to show the queues and which characters are used at each step, etc. This is tedious, but doing it will cement the ideas of the algorithm. Then, using the information about which states are distinguishable, run the state reduction algorithm to create the smallest finite state machine that accepts L(M). Show all the steps of the computation. This is tedious, but useful to do once. You can draw out the computations by hand - typesetting would be too difficult. But convert your drawings to PDF to submit.You can use the simulator to check that your computations found the correct minimum FSM

Proof. **Successive Refinement Algorithm**

F = {6}
N = {1,2,3,4,5,7}
Σ = {0,1}

Step 1:
π = {F,N} = {{6}, {1,2,3,4,5,7}}
L = {F,N} = {{6}, {1,2,3,4,5,7}}

Step 2:
π = {F,N} L = {F,N}
Let S = {F} = {6}
For w = 0;

For class F in π, since class F contains one state, it cannot be partitioned, so we ignore it for further steps of the algorithm when checking in π.

For class N in π:
δ(1,0) $\rightarrow$ $\sim$S
δ(2,0) $\rightarrow$ $\sim$S
δ(3,0) $\rightarrow$ S; So partition 3
δ(4,0) $\rightarrow$ $\sim$S

$\delta(5,0) \rightarrow$ S; So partition 5
$\delta(7,0) \rightarrow$ S; So partition 7
Therefore, N breaks down into N' and N-N'.
Where,
N' = {1,2,4}
N- N' = {3,5,7}
Update π and L.
π = {F,N',N-N'} = {{6}, {1,2,4}, {3,5,7}}
L = {F,N,N',N-N'} = {{6}, {1,2,3,4,5,7}, {1,2,4}, {3,5,7}}

For w = 1;

For class N' in π
$\delta(1,1) \rightarrow \sim$S
$\delta(2,1) \rightarrow \sim$S
$\delta(4,1) \rightarrow \sim$S
For class N-N' in π
$\delta(3,1) \rightarrow \sim$S
$\delta(5,1) \rightarrow \sim$S
$\delta(7,1) \rightarrow \sim$S
Update π and L.
π = {F,N',N-N'} = {{6}, {1,2,4}, {3,5,7}}
L = {N,N',N-N'} = {{1,2,3,4,5,7}, {1,2,4}, {3,5,7}}

Step 3
π = {F,N',N-N'} = {{6}, {1,2,4}, {3,5,7}}
L = {N,N',N-N'} = {{1,2,3,4,5,7}, {1,2,4}, {3,5,7}}
S = {N} = {1,2,3,4,5,7}
For w = 0

For class N' in π
$\delta(1,0) \rightarrow$S
$\delta(2,0) \rightarrow$S
$\delta(4,0) \rightarrow$S
All are in S so no change.

For class N-N' in π
$\delta(3,0) \rightarrow \sim$S
$\delta(5,0) \rightarrow \sim$S
$\delta(7,0) \rightarrow \sim$S
All are not in S so no change.
NO UPDATE TO π AND L.

For w = 1
For class N' in π

$\delta(1,1) \rightarrow$ S
$\delta(2,1) \rightarrow$ S
$\delta(4,1) \rightarrow$ S
All go to S so no change.

For class N-N' in π
$\delta(3,1) \rightarrow$ S
$\delta(5,1) \rightarrow$ S
$\delta(7,1) \rightarrow$ S
All go to S so no change.
NO CHANGES TO π. REMOVE N FROM L.
π = {F,N',N-N'} = {{6}, {1,2,4}, {3,5,7}}
L = {N',N-N'} = {{1,2,4}, {3,5,7}}

Step 4
π = {F,N',N-N'} = {{6}, {1,2,4}, {3,5,7}}
L = {N',N-N'} = {{1,2,4}, {3,5,7}}
S = {N} = {1,2,4}

For w = 0

For class N' in π
$\delta(1,0) \rightarrow$S
$\delta(2,0) \rightarrow$S
$\delta(4,0) \rightarrow$S
All are in S so no change.

For class N-N' in π
$\delta(3,0) \rightarrow \sim$S
$\delta(5,0) \rightarrow \sim$S
$\delta(7,0) \rightarrow \sim$S
All are not in S so no change.

For w = 1

For class N' in π
$\delta(1,1) \rightarrow \sim$S
$\delta(2,1) \rightarrow \sim$S
$\delta(4,1) \rightarrow \sim$S
All are not in S so no change.

For class N-N' in π
$\delta(3,1) \rightarrow \sim$S
$\delta(5,1) \rightarrow \sim$S
$\delta(7,1) \rightarrow \sim$S

All are not in S so no change.

NO CHANGES TO π. REMOVE N' FROM L.
π = {F,N',N-N'} = {{6}, {1,2,4}, {3,5,7}}
L = {N-N'} = {{3,5,7}}

Step 6
π = {F,N',N-N'} = {{6}, {1,2,4}, {3,5,7}}
L = {N-N'} = {{3,5,7}}

S = {N-N'} = {3,5,7}
For w = 0
For class N
For class N' in π
δ(1,0) $\rightarrow$ $\sim$S
δ(2,0) $\rightarrow$ $\sim$S
δ(4,0) $\rightarrow$ $\sim$S
All are not in S so no change.

For class N-N' in π
δ(3,0) $\rightarrow$ $\sim$S
δ(5,0) $\rightarrow$ $\sim$S
δ(7,0) $\rightarrow$ $\sim$S
All are not in S so no change.

For w = 1
For class N' in π
δ(1,1) $\rightarrow$S
δ(2,1) $\rightarrow$S
δ(4,1) $\rightarrow$S
All are in S so no change.

For class N-N' in π
δ(3,1) $\rightarrow$ S
δ(5,1) $\rightarrow$ S
δ(7,1) $\rightarrow$ S
All go to S so no change.
No change to π. Remove (N-N') from L.

π = {F,N',N-N'} = {{6}, {1,2,4}, {3,5,7}}
L = {} = {}

ALGORITHM ENDS.

State Reduction Algorithm

Reject = 0
Accept = 1

State	w = 0	w = 1	Output
1	2	3	0
2	4	5	0
3	6	7	0
4	4	5	0
5	6	7	0
6	4	5	1
7	6	7	0

Since state 3,5,7 have the same transitions and outputs; hence, they can be combined, and replace anything going to those states by the new state (3,5,7).

State	w = 0	w = 1	Output
1	2	(3,5,7)	0
2	4	(3,5,7)	0
(3,5,7)	6	(3,5,7)	0
4	4	(3,5,7)	0
6	4	(3,5,7)	1

Since state 2,and 4 are identical, we can combine the two and replace everything going to them by (2,4).

State	w = 0	w = 1	Output
1	(2,4)	(3,5,7)	0
(2,4)	(2,4)	(3,5,7)	0
(3,5,7)	6	(3,5,7)	0
6	(2,4)	(3,5,7)	1

Since 1 and (2,4) are identical, combine the two.

State	w = 0	w = 1	Output
(1,2,4)	(1,2,4)	(3,5,7)	0
(3,5,7)	6	(3,5,7)	0
6	(1,2,4)	(3,5,7)	1

SEE DIAGRAM AT THE END FOR STATE DIAGRAMS.

□

Problem 2. At the end of the notes showing that if two states are distinguishable,then they are distinguishable by a string of length at most n^2, there is a problem that states: By more closely examining the proof of the Claim show that
We only need to examine all strings of length about $n^2/2$.
That is, modify then n^2 proof. There is a small observation and modification of the proof that leads to the smaller bound. More exactly, the observation leads to a bound of (n*(n-1)/2). That might be a helpful hint.An answer that refers to the answer to problem 3, is not an answer to this question.

Proof. We found that the total strings to be tested = n^2
after comparing strings $\alpha 1$,$\alpha 2$,$\alpha 3$,$\alpha 4$,$\alpha 5$,.....αn, for machine A
and $\beta 1$,$\beta 2$,$\beta 3$,$\beta 4$,$\beta 5$,.....βn, for machine B
Lets assume our comparisons are done for ordered pairs.
When we consider n^2 strings, we also consider the case when we have (α , α) in a string and (α , α) repeated at some other point in this string. Thus, the string can always be shortened, and the computation for the repetition of the same state doesn't need to be computed. Thus, the total number of strings can be reduced to = n^2 - n. Since we have n states in every machine.

In the proof, if on computation of string on machines A and B we go to (α β) and on computation from this stage onwards we reach (β α);we can take the string between (α β) and (β α) out to shorten the string since this computation does not need to be done, since if alpha goes to accept and beta goes to reject, if on further computation it loops around to (β α) it will go to reject and accept respectively.(wlog) Thus, we know the strings are distinguishable, and we don't need to consider transitions after these states.
Thus the total number of strings can be reduced to half the size i.e. (n^2 - n)/2 or n(n -1)/2.

Another argument to prove this would be the following:
Since we know, we only need to compute the transitions for distinct pairs from n states. We can use n choose 2, since n choose 2 gives us the distinct pairs among n states. = n(n - 1)/2.

□

Problem 3. Claim 2: If two states are distinguishable, then they are distinguishable by a string of length at most n.
One way to prove this is to look closely at the successive refinement algorithm discussed in the notes on minimizing finite state machines. Recall that in the successive refinement algorithm when the algorithm splits a class C into two classes C' and C - C', it learns that every state in C' is distinguishable from every state in C - C'.

Problem 3a. Explain how the algorithm can be extended so that when a class C splits into C' and C-C', for every pair of states (q1, q2) where q1 goes into C', and q2 goes into C-C', the algorithm can quickly determine and record a string w(1,2) that distinguishes q1 from q2. Here the word quickly rules out trying out different strings to find one that distinguishes q1 and q2. Rather, the algorithm has a simple rule that creates w(1,2) at that point.

Proof. For successive refinement algorithm
We start with 2 classes, F and N.
At this point, both π and L contain both F and N classes. Because of the invariant we know all of these classes are distinguishable from each other. They are distinguished by the empty string
We take one of the classes from L, lets call it S.
Then, we start examining every state q in one of the classes in π for every class
Once we start examining all of the characters, w in Σ such that
δ (q,w) $\rightarrow$ S
Or δ (q,w) $\rightarrow \sim$ S
We partition for our class in π if some of the states lead to S while other's do not. Thus, for state S, we can determine which classes go to S, for character w. Thus, we can split the class C into C' and C-C' for a pair of states where we know on computation from C' we go to S, while from C-C' we do not, and this character w distinguishes them.

We test every class in π, including the split ones since we replace classes in pi after computation for every character c in Σ and add the split classes to L, thus we know which classes lead to the split classes to.
Thus, we start with Final, and Non-Final states that are distinguishable, and slowly break them down to find the string w(1,2) that distinguishes two states.
Every time we split a class into two, the string that distinguishes them is the string leading up to the combined state of the split states + the character w for which one of the split classes leads to S in L, and the other one doesn't for the same character and class S in L. Thus, we start with the empty string, as we always have final and non-final states in the beginning, and thus compute the characters, and therefore the string. $\square$

Problem 3b. Explain how the length of w(1,2) relates to the lengths of distinguishing strings already determined by the algorithm

Proof. Since C splits up into C' and C-C'. The strings coming up until C' and C-C' will be the same. Therefore, len(w(1,2)) = 1 + len(strings already determined).
LAST PARAGRAPH OF PREVIOUS ANSWER REITERATES THIS POINT. □

Problem 3c. Abstractly, suppose you have two sets with a total of n elements, and you pick one set that has more than one element, and split it into two sets in some way. At that point you have three sets. If there is a set with more than one element, pick one such set, and split it into two sets in some way.Continue in this way until no set has more than one element.How many splits will be executed in this process? Explain.

Proof. We have 2 sets with total of n elements.
Therefore,
Set1 = {k elements}
Set2 = {n-k elements} where k is a number, and n is the total number of states.
π = {Set1, Set2}
If we split Set 2 into more sets,we will have set2 replaced by
Set3 = {k'}
Set4 = {k-k'} where k' is a number
For the algorithm, we need at least 2 sets.
Therefore, the maximum size of a set = n-1
Hence, The maximum number of splits that can happen are (n-1) -1 = n-2
and, the maximum number of classes you can have are n. □

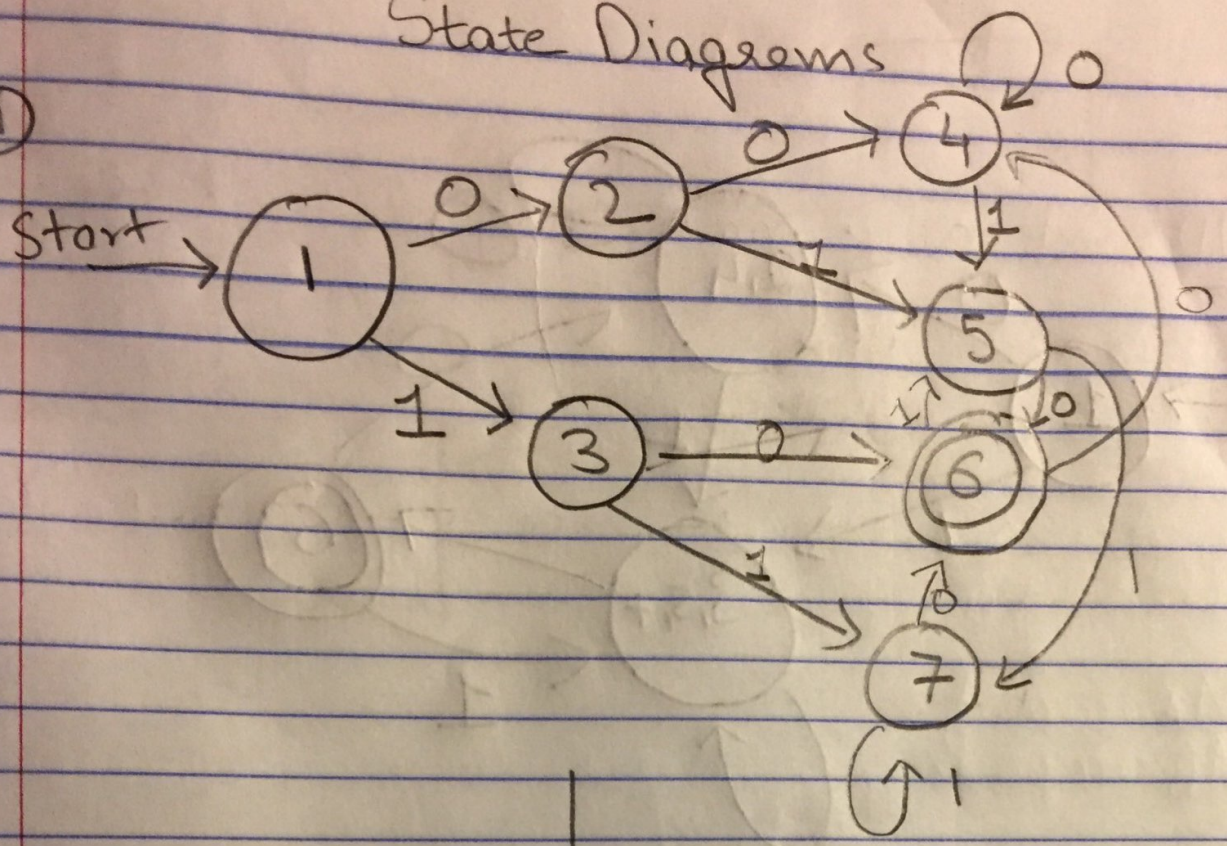
Problem 3d. Use your answers and insights from problems 3a, 3b, 3c, to create a proof of Claim 2

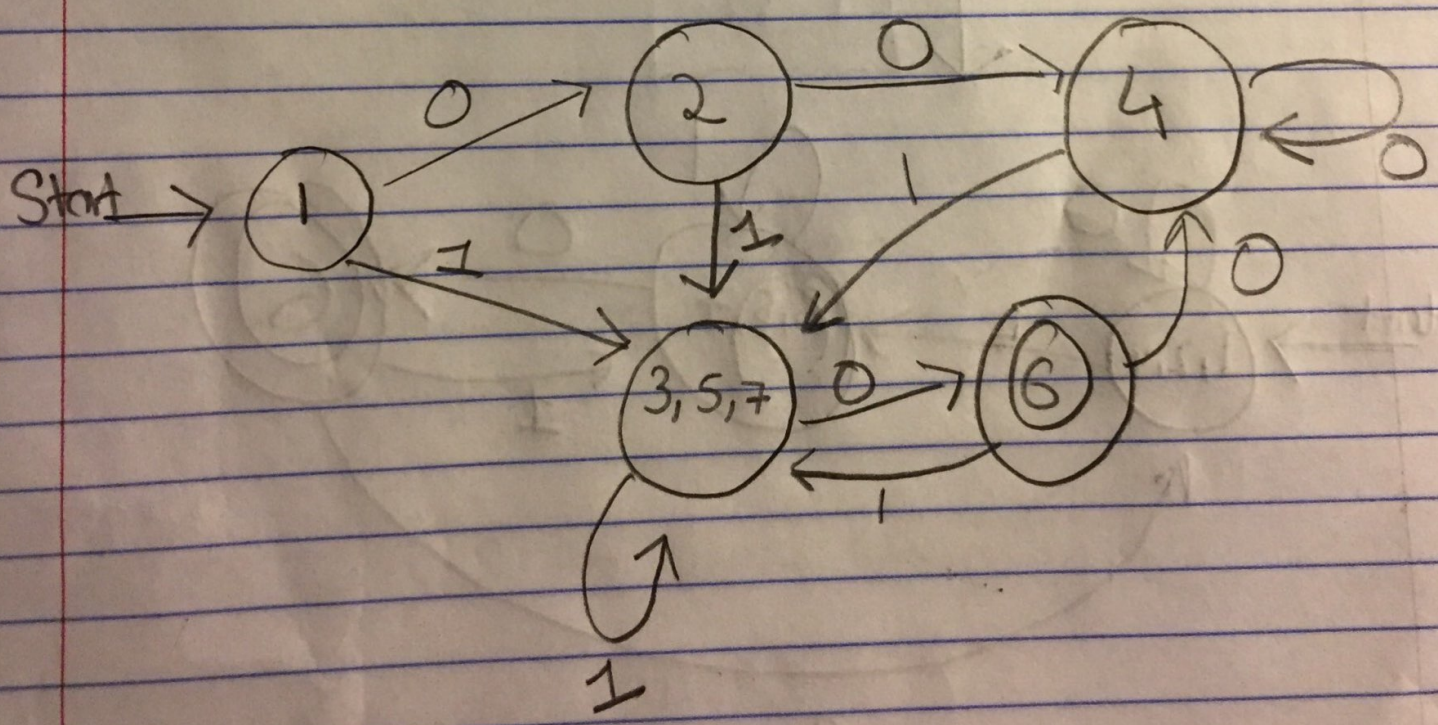
Proof. Given:
1. Max number of splits = n-2
2. Every time a split occurs, the length of a string increments by 1 start from ϵ.
3. Thus, the maximum length of string you need to examine is 1*(n-2) = (n-2). Hence, a string of length n would work too.

Hence, proved. □

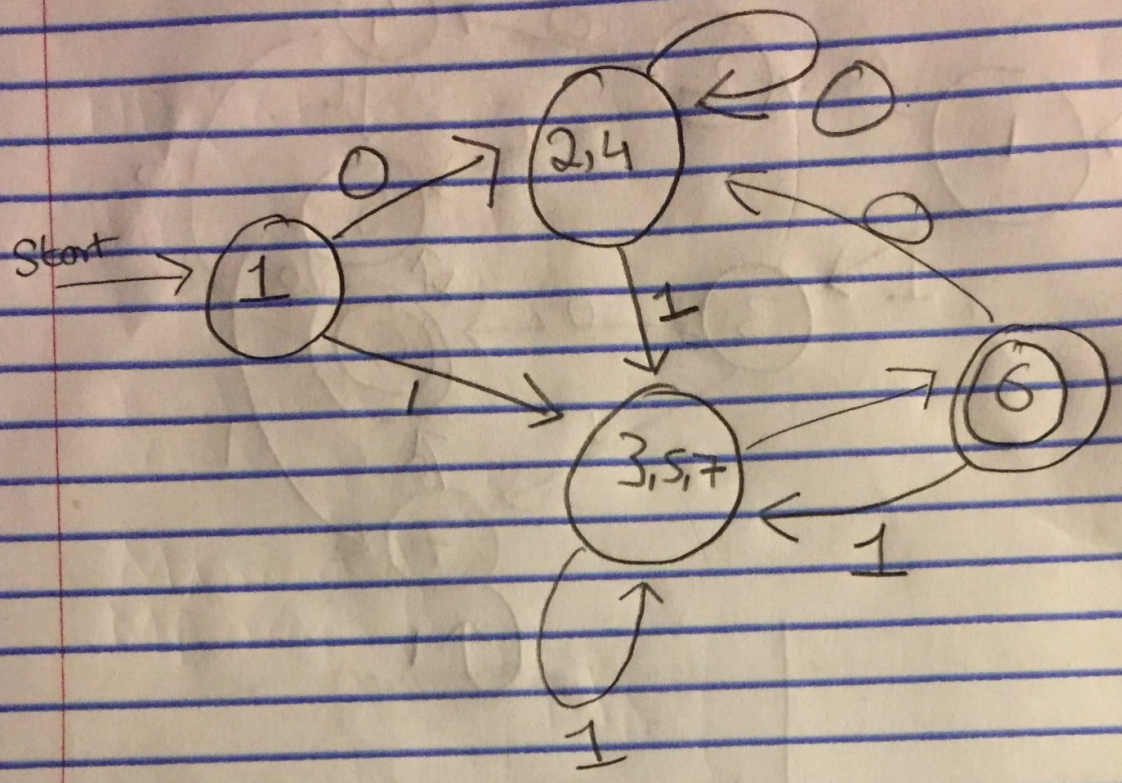
State Diagrams

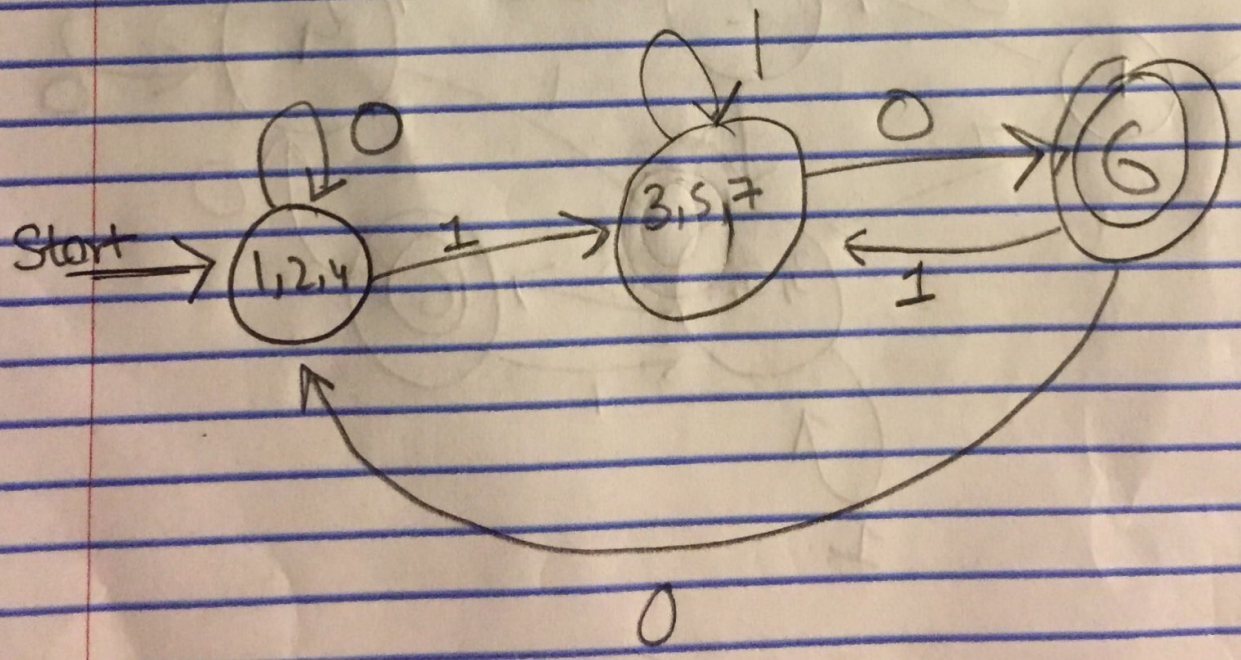
1





Scanned with CamScanner





Scanned with CamScanner